Algoritmos

2026-04-01

Ejercicio 1 de algoritmo

Consigna: Las últimas 3 crifras de mi DNI son 887 , consigna crear una variable que tenga ese número

```
DNI=887
```

Consigna: Crear un vector que tenga todos los números enteros desde 1 al 887.

```
secuencia_dni<-(1:887)
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
```

```
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810
## [811] 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828
## [829] 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846
## [847] 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864
## [865] 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882
## [883] 883 884 885 886 887
```

Ejercicio 2

calcular la suma de todos los valores del vector secuencia_dni

```r
total<-0
valor_final<-length(secuencia_dni)
for (i in 1:valor_final)
  total<-total+i
total
```

```
## [1] 393828
```

Ejercicio 3

Repetir el ejercicio anterior pero en Phyton

```python
suma = 0

for i in range(1, 887):
    suma += i

print(suma)
```

```
## 392941
```

Ejercicio 4

¿Cuanto tarda en correr el código del ejercicio 2?

```r
system.time({
  total <- 0
  valor_final <- length(secuencia_dni)

  for (i in 1:valor_final) {
    total <- total + i
  }

  print(total)
})
```

```
## [1] 393828
```

```
##    user  system elapsed
##   0.002   0.001   0.002
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```r
system.time({
  n <- length(secuencia_dni)

  # Aplicamos la fórmula de Gauss
  total_gauss <- (n * (n + 1)) / 2

  print(total_gauss)
})
```

```
## [1] 393828
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

Ejercicios del profe:

1.4 Generar un vector secuencia

```r
# --- 1. Generar secuencia con bucle 'for' ---
# Primero inicializamos el vector
system.time({
  A <- 0
  for (i in 1:50000) {
    A[i] <- (i * 2)
  }
})
```

```
##    user  system elapsed
##   0.016   0.000   0.019
```

```r
# Verificamos los resultados
head(A)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```r
system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

```r
# Verificamos los resultados
head(B)
```

```
## [1]  2  4  6  8 10 12
```

```r
tail(B)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

1.5 Implementación de una serie Fibonachi o Fibonacci

```r
# Definimos la cantidad de elementos de la serie que queremos calcular
n <- 30

# --- Método 1: Bucle 'for' con pre-asignación
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1

  for (i in 3:n) {
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.004   0.000   0.005
```

```r
# Ver los primeros resultados
print("Primeros números de la serie:")
```

```
## [1] "Primeros números de la serie:"
```

```r
head(fib, 10)
```

```
##  [1]  0  1  1  2  3  5  8 13 21 34
```

```r
# --- Método 2: Función Recursiva

fib_recursiva <- function(k) {
  if (k == 0) return(0)
  if (k == 1) return(1)
  return(fib_recursiva(k - 1) + fib_recursiva(k - 2))
}

print("Tiempo ejecución recursiva:")
```

```
## [1] "Tiempo ejecución recursiva:"
```

```r
system.time({
  # Calculamos el elemento n de forma individual
  resultado_rec <- fib_recursiva(n)
})
```

```
##    user  system elapsed
##   0.902   0.002   0.930
```

1.6 De nición matemática recurrente

```r
# --- Resolver consigna con bucle 'while' ---

# Inicializamos la serie según la definición matemática
fib <- c(0, 1)
i <- 2 # Empezamos el conteo de iteraciones/posiciones

system.time({
  # El bucle sigue mientras el último número generado sea <= 1.000.000
  while (fib[i] <= 1000000) {
    i <- i + 1
    # Definición recurrente: f(n+1) = f(n) + f(n-1)
    fib[i] <- fib[i-1] + fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.003   0.000   0.004
```

```r
# --- Resultados ---
cat("El primer número mayor a 1.000.000 es:", fib[i], "\n")
```

```
## El primer número mayor a 1.000.000 es: 1346269
```

```r
cat("Se necesitaron", i - 1, "iteraciones (pasos) para superarlo.\n")
```

```
## Se necesitaron 31 iteraciones (pasos) para superarlo.
```

```r
# Mostramos los últimos elementos para verificar
tail(fib)
```

```
## [1]  121393  196418  317811  514229  832040 1346269
```

1.7 Ordenacion de un vector por metodo burbuja

```r
# --- 1. Creamos un vector desordenado ---
# Usamos un vector de 1000 números aleatorios
set.seed(123) # Para que el resultado sea siempre el mismo
vector_desordenado <- sample(1:1000, 1000)

# --- 2. Implementación del Método Burbuja ---
bubble_sort <- function(x) {
  n <- length(x)
  # El bucle externo recorre todo el vector
  for (i in 1:(n - 1)) {
    # El bucle interno compara pares adyacentes
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        # Intercambio de posición (el "burbujeo")
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
```

```r
}

# --- 3. Medimos Performance del Burbuja vs R Nativo ---

cat("Tiempo con Método Burbuja:\n")
```

```
## Tiempo con Método Burbuja:
```

```r
system.time({
  vector_ordenado_burbuja <- bubble_sort(vector_desordenado)
})
```

```
##    user  system elapsed
##   0.074   0.000   0.077
```

```r
cat("\nTiempo con función sort() nativa de R:\n")
```

```
##
## Tiempo con función sort() nativa de R:
```

```r
system.time({
  vector_ordenado_R <- sort(vector_desordenado)
})
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

1.8 La penitencia de Newton

```r
# Definimos el límite de la suma (1*10^7 para que no tarde demasiado en R)
# Nota: 1*10^10 es un número extremadamente grande para un bucle for en R
n <- 1e7

# --- MÉTODO 1: La "Penitencia" (Bucle For / Fuerza Bruta) ---
# Este método suma uno por uno, como quería el profesor.
system.time({
  suma_iterativa <- 0
  for (i in 1:n) {
    suma_iterativa <- suma_iterativa + i
  }
})
```

```
##    user  system elapsed
##   0.170   0.001   0.171
```

```r
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```r
# --- MÉTODO 2: La "Estrategia de Newton/Gauss" (Fórmula Analítica) ---
# Usamos la suma de una progresión aritmética: (n * (n + 1)) / 2
system.time({
  suma_analitica <- (n * (n + 1)) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

```r
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

Graficos de violin

```r
# 2.0 Comparación de Performance con Gráficos de Violín
# Instalamos y cargamos la librería necesaria
if(!require(microbenchmark)) install.packages("microbenchmark")
```

```
## Loading required package: microbenchmark
```

```r
if(!require(ggplot2)) install.packages("ggplot2")
```

```
## Loading required package: ggplot2
```

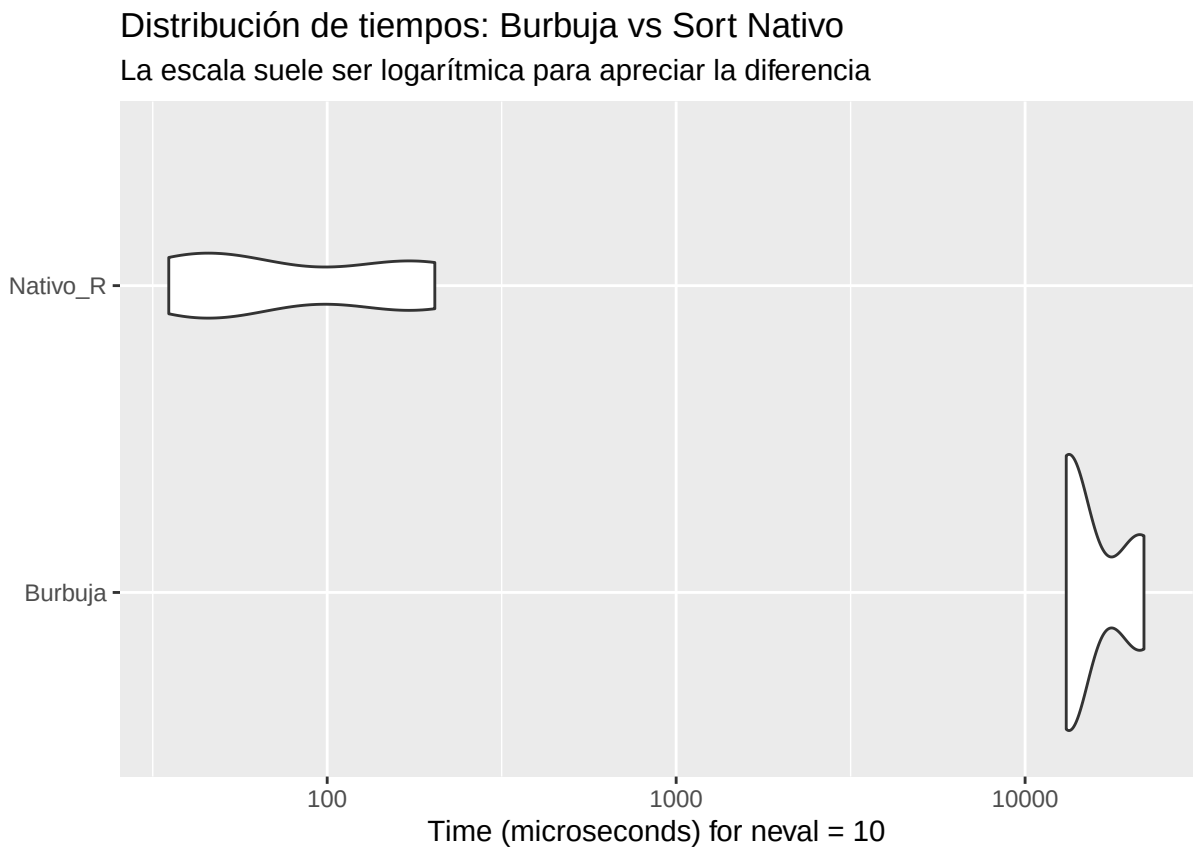
```r
library(microbenchmark)
library(ggplot2)

# --- 2.1 Comparación: Burbuja vs Sort Nativo ---
set.seed(123)
vector_prueba <- sample(1:500, 500)

# Usamos microbenchmark para ejecutar los métodos varias veces y comparar
comparativa_sort <- microbenchmark(
  Burbuja = bubble_sort(vector_prueba),
  Nativo_R = sort(vector_prueba),
  times = 10 # Lo corremos 10 veces para tener una muestra estadística
)

# Graficamos el comportamiento (Gráfico de Violín)
autoplot(comparativa_sort) +
  labs(title = "Distribución de tiempos: Burbuja vs Sort Nativo",
       subtitle = "La escala suele ser logarítmica para apreciar la diferencia",
       y = "Tiempo de ejecución", x = "Algoritmo")
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
##   Please report the issue at
##   <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



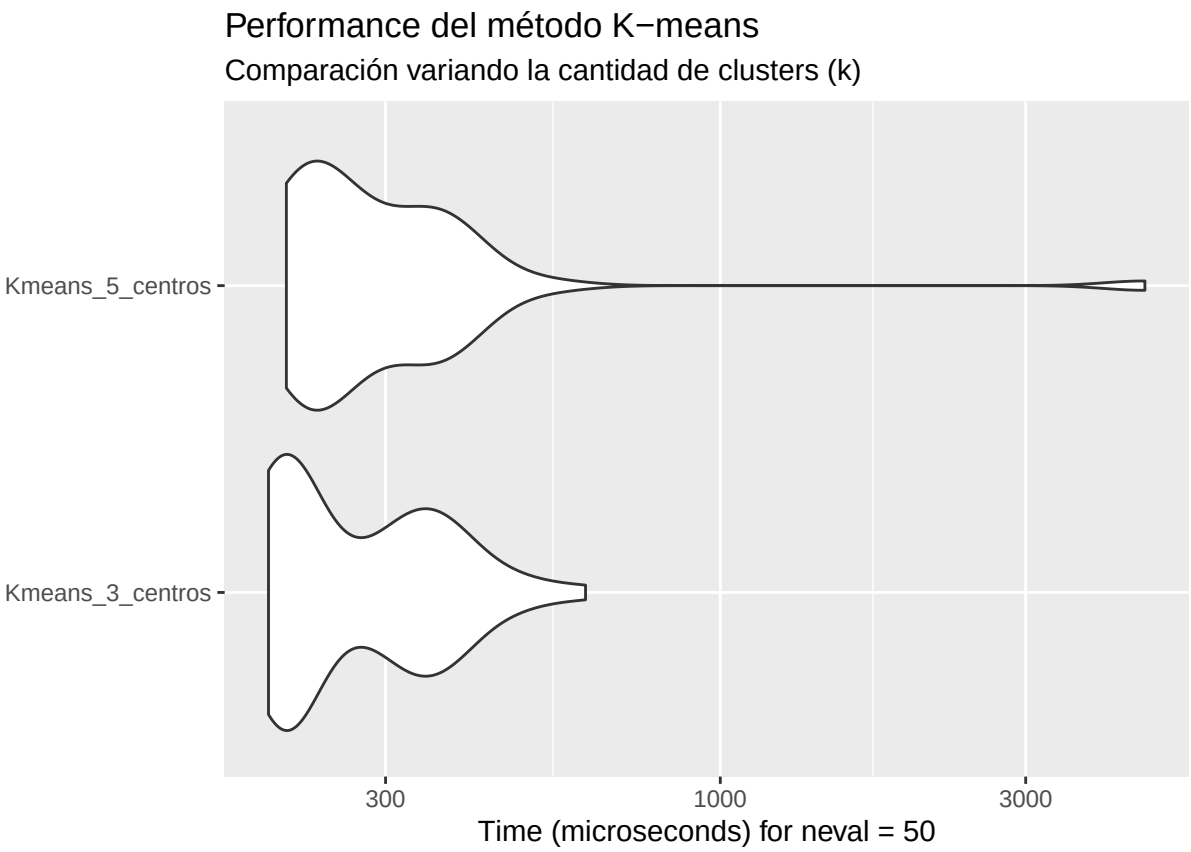

```r
# --- 2.2 K-means: Introducción Teórica y Medición ---
```

Introducción Teórica a K-means:

El algoritmo de K-means es un método de agrupamiento (clustering) que particiona un conjunto de n observaciones en k grupos. Funciona de forma iterativa asignando cada punto al "centroide" más cercano y recalculando dichos centros hasta que las posiciones se estabilizan. Es ampliamente utilizado en minería de datos para encontrar patrones no visibles a simple vista.

```r
# Medimos el rendimiento de kmeans con un dataset interno de R (iris)
comparativa_kmeans <- microbenchmark(
  Kmeans_3_centros = kmeans(iris[, 1:4], centers = 3),
  Kmeans_5_centros = kmeans(iris[, 1:4], centers = 5),
  times = 50
)

# Graficamos los resultados de K-means
autoplot(comparativa_kmeans) +
  labs(title = "Performance del método K-means",
       subtitle = "Comparación variando la cantidad de clusters (k)",
       y = "Tiempo de ejecución", x = "Configuración")
```



R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

primeros comandos de de R

R es un lenjuage muy parecido al de matlab y trabaja con variables que en general pueden ser matrices.

```
a=38
 a=40
 b=60
 c=b-a
 c
```

```
## [1] 20
```

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

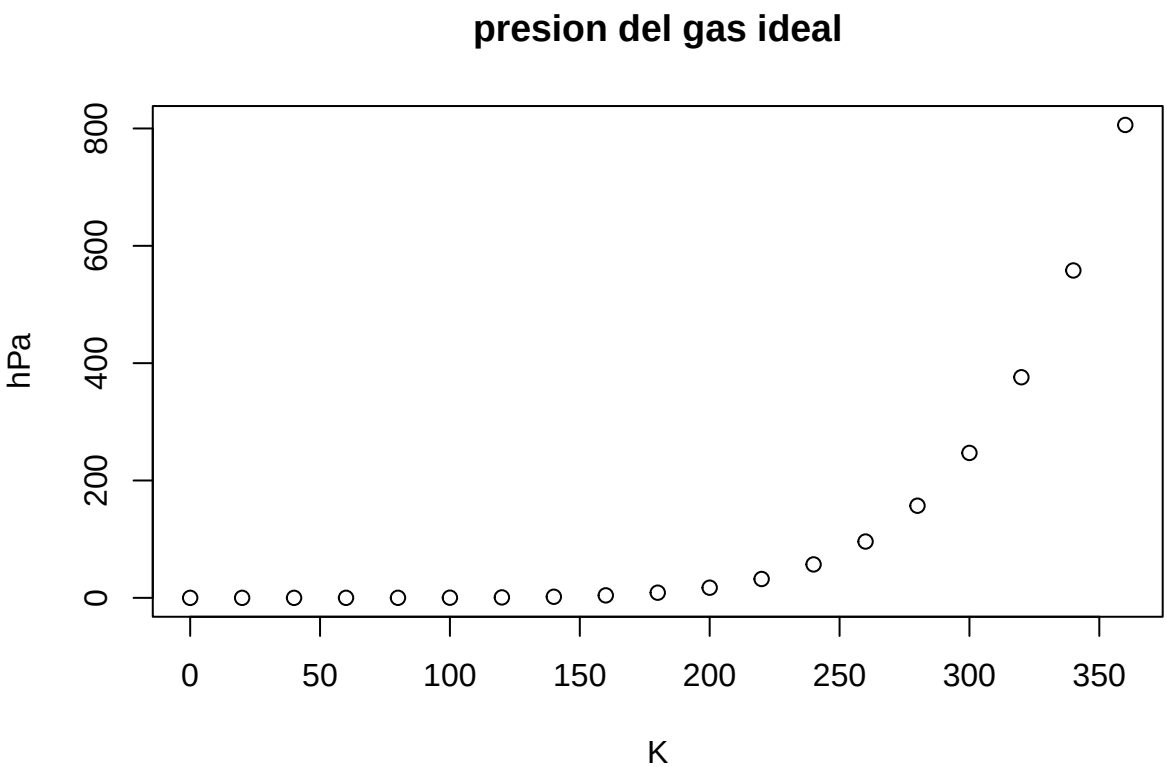
Uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ despues de el comando de informacion especifico.

Including Plots

You can also embed plots, for example:

```
plot(pressure, main="presion del gas ideal", ylab="hPa", xlab="K")
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

##Comando plot Es un comando que permite graficar cualquier fuente de datos que le asignamos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir ##plot## en la consola, lo cual abrira una ventana que explicara en detalle el comando.

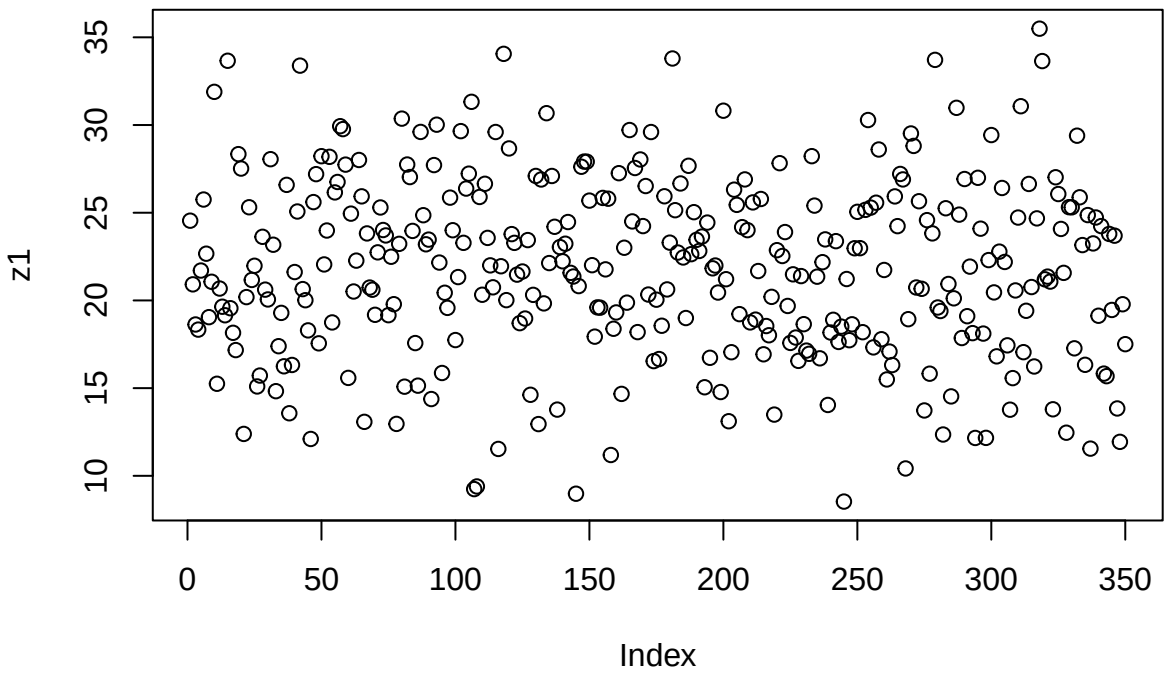
funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

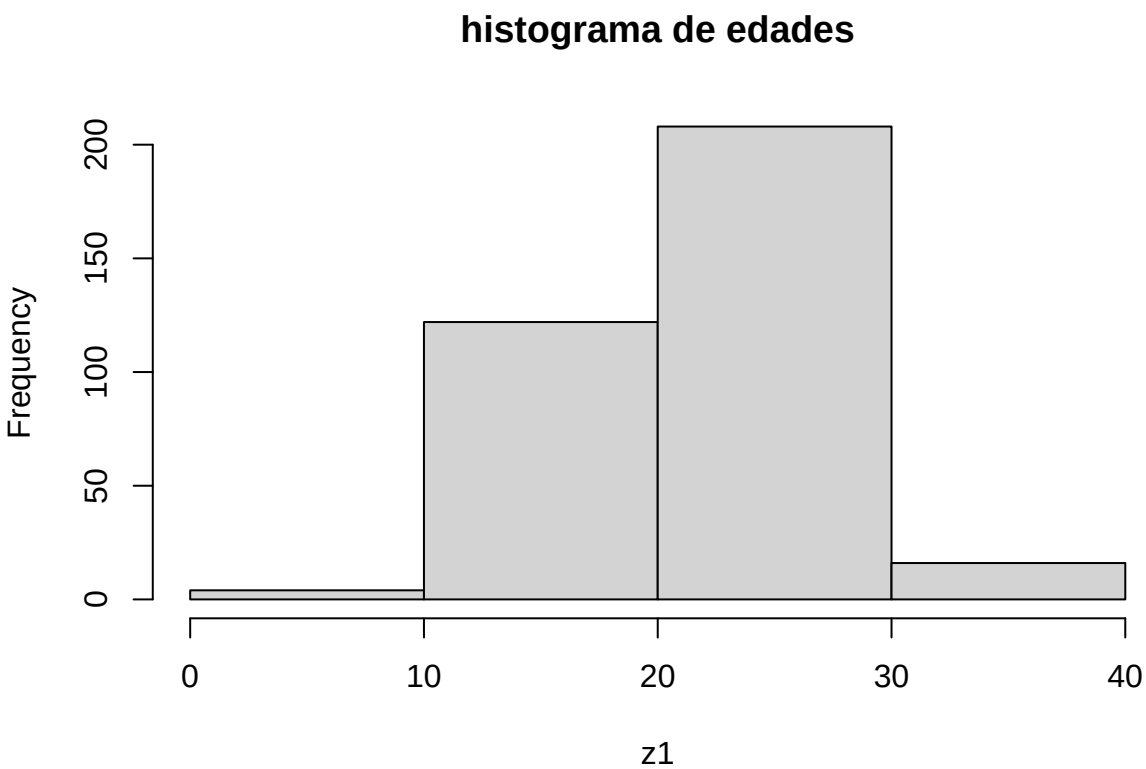
```
##   [1] 24.547046 20.916746 18.627787 18.336861 21.703351 25.752631 22.664661
##   [8] 19.046107 21.064085 31.890922 15.249269 20.677397 19.640506 19.154069
##  [15] 33.663597 19.554778 18.154379 17.169005 28.340250 27.516662 12.388475
##  [22] 20.189790 25.311236 21.167285 21.975124 15.095566 15.717366 23.626229
##  [29] 20.614997 20.054945 28.051970 23.174583 14.826608 17.391389 19.289520
##  [36] 16.243937 26.587787 13.566714 16.320736 21.618207 25.070007 33.385532
##  [43] 20.646159 20.012288 18.291457 12.102234 25.605494 27.200039 17.552892
```

```
##  [50] 28.222487 22.051297 23.985882 28.186617 18.744677 26.157581 26.752516
##  [57] 29.932604 29.767815 27.742233 15.583662 24.947276 20.508170 22.268906
##  [64] 28.006541 25.932761 13.077927 23.819986 20.734076 20.610141 19.180656
##  [71] 22.739799 25.298580 24.014593 23.704846 19.158204 22.489817 19.785465
##  [78] 12.961302 23.225454 30.365039 15.083529 27.748023 27.032338 23.962853
##  [85] 17.569034 15.141229 29.606484 24.857752 23.199904 23.467219 14.378097
##  [92] 27.715735 30.019055 22.158711 15.862080 20.437656 19.575465 25.859911
##  [99] 24.002223 17.740744 21.330526 29.651007 23.282871 26.377449 27.226416
## [106] 31.323078  9.242963  9.393800 25.897605 20.323374 26.648633 23.564555
## [113] 21.998979 20.746761 29.601376 11.533717 21.947792 34.056730 20.015972
## [120] 28.664713 23.780274 23.281306 21.470784 18.704589 21.648697 18.967333
## [127] 23.440353 14.621315 20.317810 27.096121 12.950200 26.894618 19.830610
## [134] 30.674184 22.128855 27.093999 24.197816 13.775721 23.042133 22.231535
## [141] 23.232826 24.469138 21.572468 21.362477  8.980686 20.816728 27.621075
## [148] 27.915057 27.910928 25.691111 22.009301 17.934401 19.602522 19.587020
## [155] 25.843606 21.769889 25.799351 11.188705 18.384838 19.314617 27.253984
## [162] 14.670885 23.004087 19.873026 29.713821 24.503536 27.549043 18.193790
## [169] 28.035580 24.237649 26.521056 20.336047 29.600494 16.540747 20.041519
## [176] 16.657226 18.556485 25.938227 20.627501 23.294009 33.792138 25.144151
## [183] 22.730088 26.666922 22.449012 18.995559 27.675614 22.637021 25.031058
## [190] 23.454684 22.820296 23.640303 15.050185 24.438159 16.729310 21.830144
## [197] 21.988782 20.449691 14.776952 30.815667 21.209125 13.111963 17.047380
## [204] 26.314867 25.448109 19.217685 24.176801 26.898572 24.006198 18.746932
## [211] 25.587844 18.905173 21.675375 25.782912 16.928852 18.539646 18.014236
## [218] 20.202642 13.491175 22.865544 27.824276 22.531777 23.897885 19.684375
## [225] 17.577228 21.487230 17.883929 16.553940 21.387443 18.646005 17.139677
## [232] 16.956346 28.224555 25.405354 21.353474 16.698759 22.180946 23.479448
## [239] 14.039304 18.169934 18.895705 23.378356 17.632402 18.506781  8.532592
## [246] 21.220545 17.733007 18.639130 22.974371 25.051693 22.979236 18.188296
## [253] 25.163053 30.283620 25.291636 17.322690 25.571161 28.603449 17.787480
## [260] 21.736036 15.494737 17.083769 16.315142 25.929585 24.235109 27.210158
## [267] 26.899081 10.420382 18.931136 29.518153 28.807990 20.736955 25.654818
## [274] 20.662051 13.726148 24.576654 15.824037 23.820730 33.714015 19.595847
## [281] 19.395981 12.355811 25.254413 20.936784 14.527394 20.119399 30.981009
## [288] 24.891421 17.856005 26.921633 19.101573 21.926683 18.136193 12.158752
## [295] 26.990563 24.090892 18.109134 12.163260 22.300137 29.427517 20.456678
## [302] 16.811848 22.781683 26.409789 22.193182 17.439908 13.771016 15.571438
## [309] 20.570145 24.724133 31.071441 17.049819 19.409837 26.639740 20.763296
## [316] 16.234161 24.677776 35.491975 33.645391 21.203923 21.357355 21.074159
## [323] 13.794046 27.022101 26.064686 24.074883 21.570705 12.458955 25.316260
## [330] 25.308429 17.269577 29.392165 25.879115 23.155341 16.328828 24.865632
## [337] 11.550790 23.249258 24.727802 19.127399 24.237295 15.827328 15.682342
## [344] 23.791294 19.457288 23.692420 13.840754 11.937838 19.782362 17.505638
```

```r
plot(z1)
```




```r
hist(z1, main="histograma de edades", breaks=2)
```

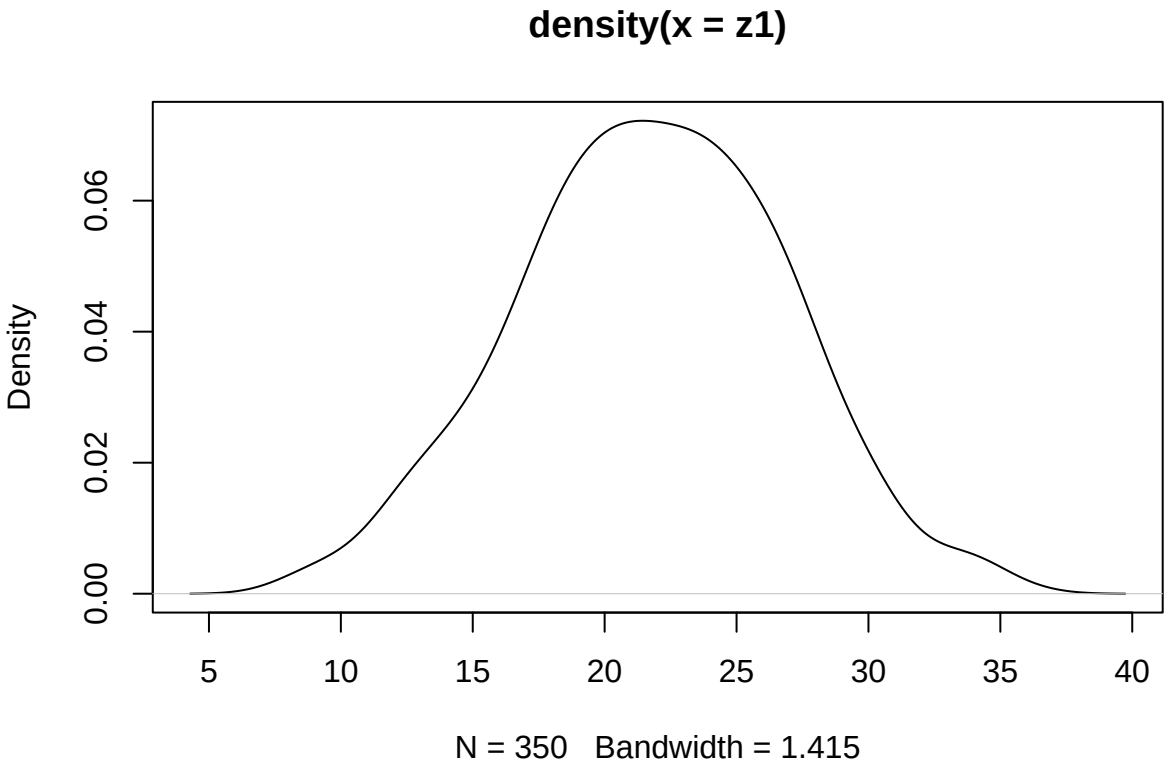



```r
density(z1)
```

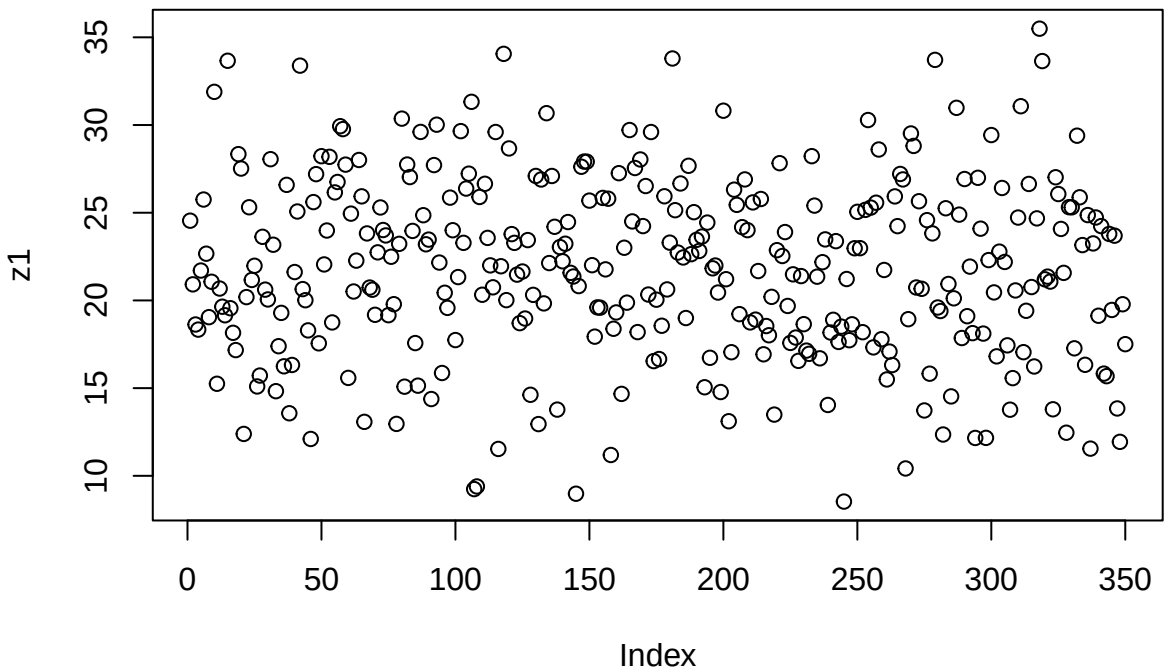
```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.415
```

```
##
##        x                y
##  Min.   : 4.289   Min.   :9.659e-06
##  1st Qu.:13.150   1st Qu.:3.785e-03
##  Median :22.012   Median :1.886e-02
##  Mean   :22.012   Mean   :2.816e-02
##  3rd Qu.:30.874   3rd Qu.:5.506e-02
##  Max.   :39.736   Max.   :7.219e-02
```

```r
plot(density(z1))
```




```r
plot(z1)
```




```r
w1<-length(z1)
w1
```

```
## [1] 350
```

```r
x1<-(500:849)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```